MAKE MY MARRIAGE

System Design & Architecture

Modular monolith architecture for the Make My Marriage MVP

Project documentation | Version 1.0 | September 2026

Document Guide

This document is organized into the following sections:

1. Architecture Goals
2. High-Level Architecture
3. Application and Data Layers
4. Authentication and Authorization
5. Media Architecture
6. Public Experiences
7. Deployment and Security
8. Scalability and Implementation Guidance

1. Architecture Goals

- Fast MVP development with low operational complexity.
- India-focused deployment and data residency where supported.
- Secure multi-user wedding collaboration.
- Large-scale photo handling without sending file bytes through the application server.
- Clear separation between private planning, token-based guest access and public content.
- Easy future scaling without premature microservices.

2. Architecture Style

The initial system uses a modular monolith: one deployable backend application organized into business modules. This keeps development, testing and deployment simple while preserving clear service boundaries.

```
Application
|- Authentication
|- Weddings
|- Members
|- Events
|- Guests / RSVP
|- Tasks
|- Expenses
|- Shopping
|- Vendors
|- Invitations
|- Wedding Website
|- Gallery / Media
|- QR Upload
|- Livestream
`- Notifications
```

3. Technology Decisions

Area	Decision	Reason
Frontend	Next.js	Supports authenticated application UI and SEO-friendly public pages.
Backend	Node.js modular monolith	Matches product velocity while keeping domain modules separated.
Database	MongoDB Atlas	Flexible document model suited to wedding, event and guest metadata.
Media	Amazon S3	Durable object storage for photos, videos and receipts.
CDN	Amazon CloudFront	Fast controlled delivery of media while S3 remains private.
Authentication	Google OAuth initially	Simple identity experience for couples and family members.
Queue	Amazon SQS later	Async processing for image jobs, emails and bulk work.
Email	Amazon SES later	Transactional invitations and notifications.
Identity option	Amazon Cognito later	Possible centralized production identity service.

4. High-Level Architecture

```
Users
  |
CloudFront
  |----------------------> S3 (photos/videos)
  |
Next.js Web Application
  |
Node.js Modular Monolith
  |
MongoDB Atlas
  |
External integrations (Google, maps, email, livestream links)

Future async path:
Node.js -> SQS -> Worker -> S3 / MongoDB / SES
```

5. Frontend Architecture

```
/                    Landing page
/demo                Ashu and Priya demo
/login               Google authentication
/app/dashboard       Private dashboard
/app/events          Event management
/app/guests          Guest management
/app/tasks           Tasks
/app/expenses        Expenses
/app/shopping        Shopping
/app/vendors         Vendors
/app/invitations     Invitations
/app/photos          Gallery/media
/app/livestream      Livestream settings
/app/members         Member management
/app/settings        Wedding settings
/w/[weddingSlug]     Public wedding website
/rsvp/[token]        Guest RSVP
/gallery/[token]     Shared gallery
/upload/[token]      Guest/QR upload
```

6. Backend Module Structure

```
server/
  modules/
    auth/
    weddings/
    members/
    events/
    guests/
    rsvp/
    tasks/
    expenses/
    shopping/
    vendors/
    invitations/
    website/
    media/
    gallery/
    livestream/
  middleware/
  database/
  integrations/
  security/
  shared/
```

Each module owns routing, validation, services, authorization rules and data-access logic. Business logic should not live in React components or thin route handlers.

7. Data Architecture

MongoDB stores structured data and media metadata. S3 stores the actual photo/video bytes. The weddingId field is the main tenant boundary for wedding-owned data.

```
Wedding
|- Members
|- Events
|- Guests -> Event Guests / RSVP
|- Tasks
|- Expenses
|- Shopping Items
|- Wedding Vendors -> Event Vendors
|- Invitations
|- Albums -> Media -> S3
|- Upload Sessions
|- Share Links
|- Livestreams
`- Activity Logs
```

8. Authentication and Authorization

- Google authentication answers who the user is.
- Wedding membership answers what wedding the user can access.
- Role authorization determines what actions are allowed.
- MVP roles are Admin and Member.
- Frontend visibility is never treated as authorization.
- Every protected resource is resolved to a wedding and checked against wedding membership.

```
Authenticate user
  -> Resolve requested resource
  -> Resolve weddingId
  -> Verify ACTIVE wedding membership
  -> Verify required role
  -> Execute operation
```

9. Media Storage and Upload

Large files should never be proxied through the Node.js application. The backend creates short-lived presigned S3 upload URLs after validating the user or guest token.

```
Browser
  1. Request upload authorization
  v
Node.js API
  2. Validate member or guest token
  3. Create media record + presigned URL
  v
Browser
  4. Upload file directly
  v
Amazon S3
  5. Completion callback/API confirmation
  v
Node.js / Worker
  6. Verify object, generate derivatives, update metadata
  v
MongoDB
```

10. S3 and CloudFront

```
weddings/{weddingId}/
  originals/
  thumbnails/
  optimized/
  videos/
  receipts/
```

- S3 buckets remain private.
- CloudFront is the normal media delivery path.
- Gallery grids should use thumbnails; fullscreen views should use optimized images; originals should be delivered only when necessary.

11. Guest QR Upload

```
Guest scans QR
  -> /upload/{secureToken}
  -> Validate upload session
  -> Select allowed event
  -> Request presigned upload URL
  -> Direct upload to S3
  -> Save media metadata
  -> Add to private wedding gallery
```

- No account required.
- Admin can disable uploads.
- Apply file size/type limits, per-session limits and rate limiting.

- Use random high-entropy tokens.

12. Public Wedding Website

> **Separation rule:** Private dashboard objects must never be returned wholesale and then hidden on the frontend. Public APIs must explicitly select safe fields.

- Unique wedding slug such as /w/ashu-priya.
- Public sections are configurable.
- Initial website theme is Classic.
- RSVP, shared gallery and guest upload remain separate tokenized capabilities.

13. Livestream

The MVP stores external livestream URLs per event. It does not build or operate video streaming infrastructure. The wedding website can expose a Watch Live action for active or scheduled streams.

14. Deployment

- Use separate Development, Staging and Production environments.
- Prefer deployment in an India AWS region where required services are available.
- Use separate databases and media prefixes/buckets for environments.
- Store production secrets in managed secret storage rather than Git or plaintext configuration.

```
CloudFront
  |-----------------> S3
  v
Next.js
  v
Node.js App
  v
MongoDB Atlas (India deployment where supported)

Future: SQS workers + SES
```

15. Security Requirements

- Server-side authorization for every private operation.
- High-entropy tokens for member invitations, RSVP, gallery sharing and guest uploads.
- Rate limiting on authentication and public token endpoints.
- Upload MIME, extension and size validation.
- No public S3 buckets.
- Avoid logging authentication tokens, secrets, signed URLs and unnecessary guest PII.
- Use HTTPS everywhere.

16. Backups and Durability

- MongoDB Atlas backups protect structured planning data.
- S3 provides media durability; versioning/lifecycle policies can be added as cost allows.
- Database backups alone are insufficient because photos and videos live in object storage.

17. Demo Architecture

The canonical Ashu and Priya demo wedding is read-only for anonymous visitors. Apparent write actions can be simulated locally or applied to an isolated session copy that resets automatically.

18. Scalability Strategy

Stage	Architecture Evolution
Stage 1	Next.js + Node.js + MongoDB + S3 + CloudFront.
Stage 2	Add SQS, workers, SES, caching, observability and async image processing.
Stage 3	Extract only proven bottlenecks such as media processing, notifications or vendor search into services.

19. Recommended Repository Structure

```
make-my-marriage/
  apps/
    web/
  server/
    modules/
    middleware/
    database/
    integrations/
  packages/
    shared-types/
    validation/
    ui/
  infrastructure/
  scripts/
  docs/
```

20. Access Model Summary

Access Type	Examples
Private authenticated	Dashboard, guests, expenses, tasks, members, vendor management.
Authenticated member	Family collaboration within an assigned wedding.
Token-based	Member invite, RSVP, guest upload, shared gallery.
Public	Landing page, selected wedding website sections, demo.